DBMS

25 March 2026 22:22

1. DBMS and Its Applications

A **Database Management System (DBMS)** is software that enables users to create, store, manage, retrieve, and manipulate data in a structured and efficient manner. It acts as an interface between the user and the database.

Key characteristics of a DBMS:

- Data independence (logical and physical)
- Reduced data redundancy
- Concurrent access by multiple users
- Data security and authorization
- Backup and recovery mechanisms
- Data integrity enforcement

Applications of DBMS:

Domain	Application	Example
Banking	Account management, transactions	Core banking systems
Airlines	Reservations, schedules	IRCTC, booking systems
Universities	Student records, grades, courses	University ERP systems
Telecom	Call records, billing, subscriber data	Airtel, Jio CRM
Healthcare	Patient records, prescriptions	Hospital Management Systems
E-commerce	Products, orders, customers	Amazon, Flipkart
HR Systems	Payroll, employee records	SAP HR

DBMS vs File System:

Feature	File System	DBMS
Data redundancy	High	Low (controlled)
Data consistency	Difficult to maintain	Enforced by DBMS
Concurrent access	Not supported well	Full support
Data security	OS-level only	Role-based access
Query capability	None	Powerful SQL queries
Recovery	Manual	Automatic

2. Integrity

Data Integrity means ensuring that the data stored in the database is **accurate, consistent, and valid** throughout its lifecycle.

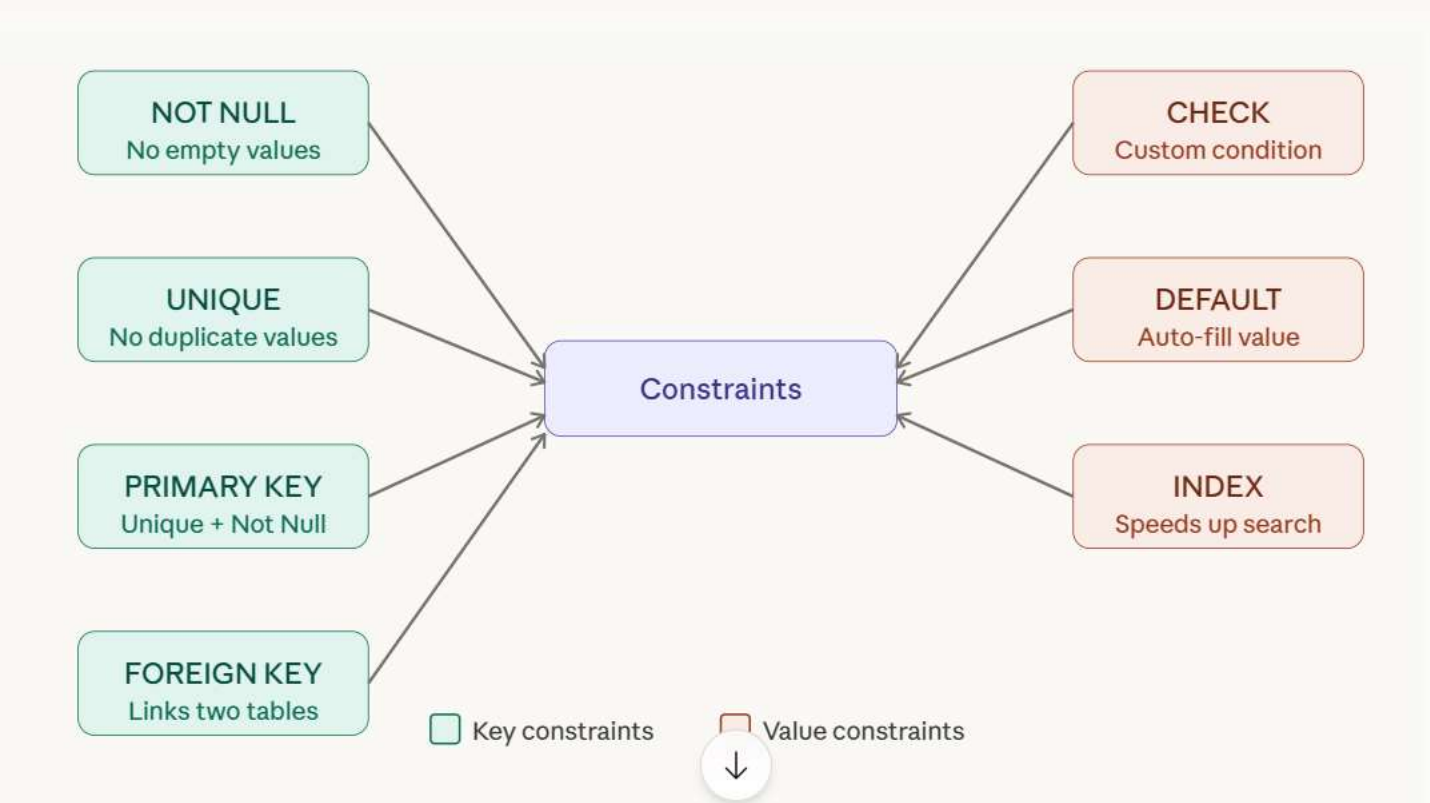
There are four main types of integrity:

1. Entity Integrity Every table must have a primary key, and the primary key value must be unique and NOT NULL.

Example: In a Student table, each student must have a unique student_id. Two students cannot share the same ID, and no student can have a NULL ID.
2. Referential Integrity A foreign key value in one table must either match a primary key in the referenced table, or be NULL. It ensures relationships between tables remain valid.
Example: If Orders table has a customer_id foreign key referencing the Customers table, you cannot insert an order for a customer that doesn't exist.
3. Domain Integrity All values in a column must belong to a defined set of valid values (data type, format, range).
Example: An age column must only accept integers between 0 and 150. A gender column might only accept 'M' or 'F'.
4. User-Defined Integrity Business rules specific to an organization that go beyond standard constraints.
Example: An employee's salary cannot be greater than the manager's salary. A student cannot register for more than 6 subjects per semester.

3. Constraints

Constraints are **rules enforced on columns** of a table to ensure the accuracy and reliability of data. They are defined at the time of table creation or can be added later.



Screen clipping taken: 26-03-2026 05:50

Detailed explanation of each constraint with SQL syntax:
1. NOT NULL — Ensures a column cannot store a NULL value.

```
CREATE TABLE Student (
  student_id INT NOT NULL,
  name VARCHAR(50) NOT NULL
);
```

2. UNIQUE — Ensures all values in a column are different. Unlike PRIMARY KEY, a table can have multiple UNIQUE constraints, and UNIQUE columns can hold NULL.
sql

```
CREATE TABLE Student (
  email VARCHAR(100) UNIQUE
);
```

3. PRIMARY KEY — A combination of NOT NULL + UNIQUE. Uniquely identifies each row. Only one primary key per table (but it can be composite).

```
CREATE TABLE Student (
  student_id INT PRIMARY KEY,
  name VARCHAR(50)
);
```

4. FOREIGN KEY — Establishes a link between two tables. The value must exist in the referenced (parent) table.

```
CREATE TABLE Orders (
  order_id INT PRIMARY KEY,
  customer_id INT,
  FOREIGN KEY (customer_id) REFERENCES Customer(customer_id)
);
```

5. CHECK — Ensures values in a column satisfy a specified condition.

```
CREATE TABLE Employee (
  age INT CHECK (age >= 18 AND age <= 60),
  salary DECIMAL CHECK (salary > 0)
);
```

6. DEFAULT — Assigns a default value when no value is specified during insertion.

```
CREATE TABLE Orders (
  order_date DATE DEFAULT GETDATE(),
  status VARCHAR(20) DEFAULT 'Pending'
);
```

4. ER Models (Entity-Relationship Model)

An **ER Model** is a conceptual data model that describes the structure of a database in terms of entities, attributes, and relationships between entities. It was introduced by **Peter Chen in 1976**.

Components of an ER Diagram:

Symbol	Represents	Example
Rectangle	Entity	Student, Employee
Ellipse	Attribute	Name, Age, ID
Diamond	Relationship	Enrolls, Works_In
Double Rectangle	Weak Entity	Dependent (depends on Employee)
Double Ellipse	Multivalued Attribute	Phone numbers
Dashed Ellipse	Derived Attribute	Age (derived from DOB)
Underlined Attribute	Key Attribute	Student_ID

Screen clipping taken: 26-03-2026 05:53

Types of Attributes:

- **Simple** — Cannot be divided further. e.g., Age.
- **Composite** — Can be divided. e.g., Name → First Name + Last Name.
- **Multivalued** — Can have multiple values. e.g., Phone Numbers.
- **Derived** — Derived from another attribute. e.g., Age from Date of Birth.
- **Key** — Uniquely identifies an entity. e.g., Student_ID.

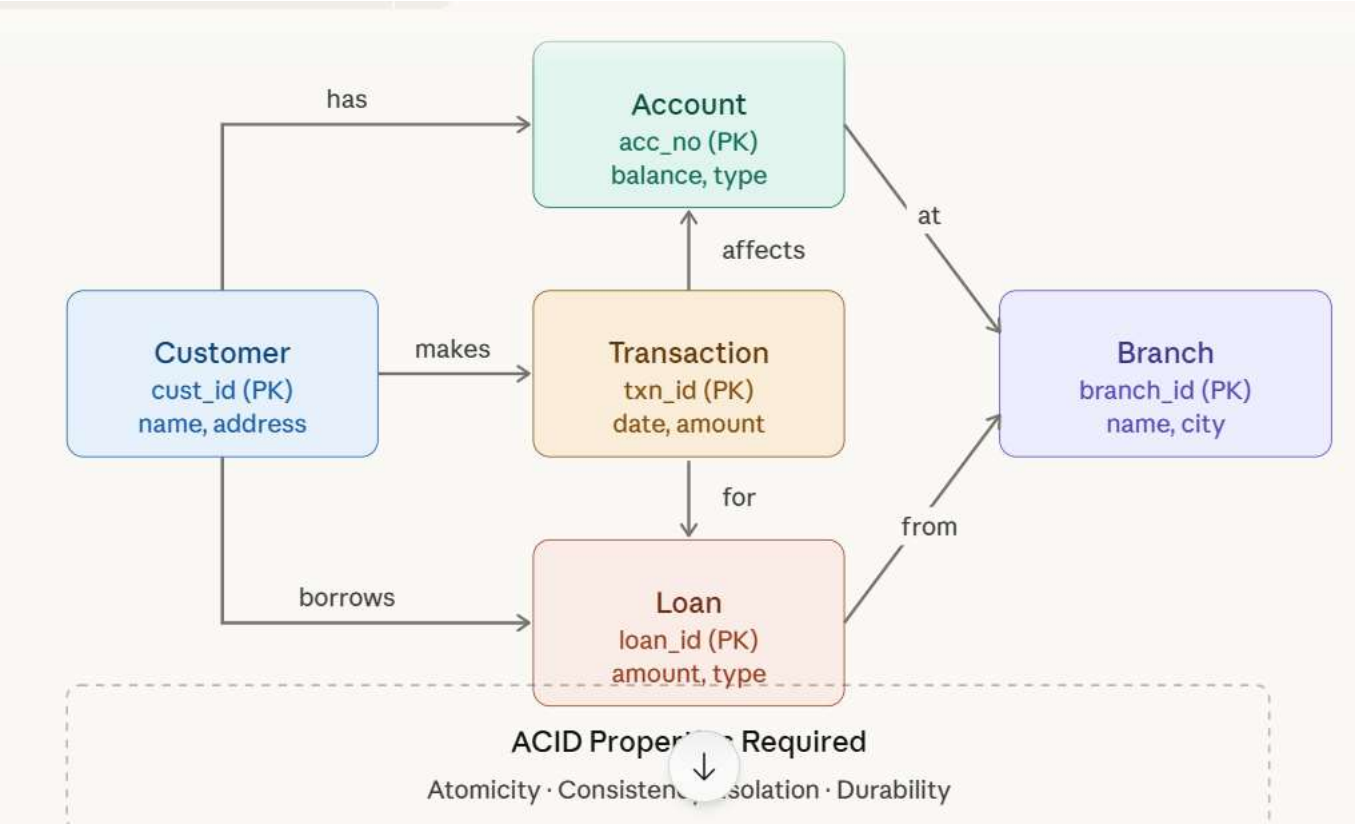
Types of Relationships (Cardinality):
One-to-One (1:1) — One entity in A is related to at most one entity in B. Example: One Person has one Passport.
One-to-Many (1:N) — One entity in A is related to many entities in B. Example: One Teacher teaches many Students.
Many-to-Many (M:N) — Many entities in A relate to many entities in B. Example: Many Students enroll in many Courses.
Participation Constraints:

- **Total participation** (double line) — Every entity must participate in the relationship. e.g., Every Employee must work in a Department.
- **Partial participation** (single line) — Not every entity must participate. e.g., Not every Employee manages a Department.

Weak Entity: An entity that cannot be uniquely identified by its own attributes alone. It depends on a strong entity. Uses a **discriminator** (partial key) combined with the parent's primary key.
Example: Dependent (weak entity) depends on Employee. A dependent is identified by (employee_id + dependent_name).

5. Banking Database Architecture

A banking system is one of the most classic DBMS applications. It requires strict ACID properties (Atomicity, Consistency, Isolation, Durability).



Screen clipping taken: 26-03-2026 05:55

Core tables in a Banking Database:

```
-- Branch table
CREATE TABLE Branch (
  branch_id   INT PRIMARY KEY,
  branch_name VARCHAR(50),
  city        VARCHAR(50),
  assets      DECIMAL(15,2)
);

-- Customer table
CREATE TABLE Customer (
  cust_id   INT PRIMARY KEY,
  name      VARCHAR(100),
  address   VARCHAR(200),
  phone     VARCHAR(15)
);

-- Account table
CREATE TABLE Account (
  acc_no    INT PRIMARY KEY,
  cust_id   INT REFERENCES Customer(cust_id),
  branch_id INT REFERENCES Branch(branch_id),
  balance   DECIMAL(12,2) CHECK (balance >= 0),
  acc_type  VARCHAR(20) DEFAULT 'Savings'
);
```

```
-- Loan table
CREATE TABLE Loan (
  loan_id   INT PRIMARY KEY,
  cust_id   INT REFERENCES Customer(cust_id),
  branch_id INT REFERENCES Branch(branch_id),
  amount    DECIMAL(12,2),
  loan_type VARCHAR(20)
);

-- Transaction table
CREATE TABLE Transaction (
  txn_id    INT PRIMARY KEY,
  acc_no    INT REFERENCES Account(acc_no),
  txn_date  DATE DEFAULT CURRENT_DATE,
  amount    DECIMAL(12,2),
  txn_type  VARCHAR(10) CHECK (txn_type IN ('CREDIT','DEBIT'))
);
```

ACID Properties in Banking:

- **Atomicity** — A fund transfer either completes fully (debit + credit) or not at all. No partial transactions.
- **Consistency** — Total money in the system remains constant before and after a transfer.
- **Isolation** — Two transactions happening simultaneously don't interfere with each other.
- **Durability** — Once a transaction is committed, it persists even if the system crashes.

6. DDL — Data Definition Language

DDL commands are used to **define and modify the structure** of the database schema. These commands affect the schema, not the data. DDL changes are **auto-committed** (cannot be rolled back in most RDBMS).

DDL Commands:

CREATE

Used to create a new database, table, view, index, or other objects.

```
-- Create a database
CREATE DATABASE CollegeDB;

-- Create a table
CREATE TABLE Student (
  student_id   INT         PRIMARY KEY,
  name         VARCHAR(100) NOT NULL,
  dob          DATE,
  branch       VARCHAR(50)  DEFAULT 'CSE',
  cgpa         DECIMAL(3,2) CHECK (cgpa BETWEEN 0.0 AND 10.0),
  email        VARCHAR(100) UNIQUE
);
```

ALTER

Used to add, delete, or modify columns in an existing table.

```
-- Add a new column
ALTER TABLE Student ADD phone VARCHAR(15);

-- Modify column data type
ALTER TABLE Student MODIFY cgpa DECIMAL(4,2);

-- Rename a column (MySQL)
ALTER TABLE Student RENAME COLUMN phone TO mobile;

-- Drop a column
ALTER TABLE Student DROP COLUMN dob;

-- Add a constraint
ALTER TABLE Student ADD CONSTRAINT chk_cgpa CHECK (cgpa >= 0);
```

DROP

Permanently deletes the entire table or database including all data and structure.
sql

```
-- Drop a table
DROP TABLE Student;

-- Drop a database
DROP DATABASE CollegeDB;
```

TRUNCATE

Removes **all rows** from a table but keeps the table structure intact. Faster than DELETE as it doesn't log individual row deletions.

```
TRUNCATE TABLE Student;
```

RENAME

Renames a database object.

```
-- Rename table (Oracle/MySQL syntax)
ALTER TABLE Student RENAME TO Learner;

-- SQL Server syntax
EXEC sp_rename 'Student', 'Learner';
```

DDL vs DML comparison:

Feature	DDL	DML
Works on	Schema/Structure	Data/Rows
Auto-commit	Yes	No (manual commit)
Rollback	Not possible	Possible
Examples	CREATE, ALTER, DROP	INSERT, UPDATE, DELETE

7. DML — Data Manipulation Language

DML commands are used to **insert, update, delete, and retrieve data** from tables. DML operations are NOT auto-committed — they can be rolled back.

INSERT

Adds new rows to a table.

```
-- Insert all columns
INSERT INTO Student VALUES (101, 'Ravi Kumar', '2002-05-10', 'CSE', 8.5, 'ravi@email.com');

-- Insert specific columns
INSERT INTO Student (student_id, name, branch)
VALUES (102, 'Priya Sharma', 'ECE');

-- Insert multiple rows
INSERT INTO Student VALUES
  (103, 'Arjun', '2001-11-20', 'MECH', 7.8, 'arjun@email.com'),
  (104, 'Sneha',  '2002-03-15', 'CSE',  9.1, 'sneha@email.com');
```

SELECT

Retrieves data from one or more tables. This is the most used DML command.

```
-- Select all columns
SELECT * FROM Student;

-- Select specific columns
SELECT name, cgpa FROM Student;

-- With WHERE clause
SELECT * FROM Student WHERE branch = 'CSE' AND cgpa > 8.0;

-- With ORDER BY
SELECT name, cgpa FROM Student ORDER BY cgpa DESC;

-- With GROUP BY and HAVING
SELECT branch, AVG(cgpa) AS avg_cgpa
FROM Student
GROUP BY branch
HAVING AVG(cgpa) > 7.5;

-- JOIN example
SELECT s.name, c.course_name
FROM Student s
JOIN Enrollment e ON s.student_id = e.student_id
JOIN Course c     ON e.course_id  = c.course_id;

-- Subquery
SELECT name FROM Student
WHERE cgpa > (SELECT AVG(cgpa) FROM Student);
```

UPDATE

Modifies existing data in a table.

```
-- Update a single row
UPDATE Student SET cgpa = 9.0 WHERE student_id = 101;

-- Update multiple rows
UPDATE Student SET branch = 'IT' WHERE branch = 'CSE' AND cgpa < 6.0;

-- Update with subquery
UPDATE Student SET cgpa = cgpa + 0.5
WHERE student_id IN (SELECT student_id FROM TopPerformers);
```

DELETE

Removes rows from a table.

```
-- Delete a specific row
DELETE FROM Student WHERE student_id = 101;

-- Delete multiple rows
DELETE FROM Student WHERE cgpa < 5.0;

-- Delete all rows (keeps structure, can be rolled back unlike TRUNCATE)
DELETE FROM Student;
```

Important SQL Clauses Summary:

Clause	Purpose	Example
WHERE	Filter rows	WHERE age > 20
ORDER BY	Sort results	ORDER BY name ASC
GROUP BY	Group rows	GROUP BY dept
HAVING	Filter groups	HAVING COUNT(*) > 5
DISTINCT	Remove duplicates	SELECT DISTINCT city
LIMIT	Limit rows returned	LIMIT 10
LIKE	Pattern matching	WHERE name LIKE 'A%'
BETWEEN	Range check	WHERE age BETWEEN 18 AND 25
IN	List check	WHERE dept IN ('CSE','IT')

8. DCL — Data Control Language

DCL commands are used to **control access** to database objects. They manage permissions and user privileges.

GRANT

Gives a user specific privileges on a database object.

```
-- Grant SELECT privilege on a table
GRANT SELECT ON Student TO user1;
```

```
-- Grant multiple privileges
GRANT SELECT, INSERT, UPDATE ON Student TO user2;

-- Grant all privileges
GRANT ALL PRIVILEGES ON Student TO admin_user;

-- Grant with GRANT OPTION (user can further grant to others)
GRANT SELECT ON Student TO user3 WITH GRANT OPTION;

-- Grant on entire database (MySQL)
GRANT ALL PRIVILEGES ON CollegeDB.* TO 'user1'@'localhost';
```

REVOKE

Removes previously granted privileges from a user.

```
-- Revoke a specific privilege
REVOKE SELECT ON Student FROM user1;

-- Revoke multiple privileges
REVOKE INSERT, UPDATE ON Student FROM user2;

-- Revoke all privileges
REVOKE ALL PRIVILEGES ON Student FROM user2;
```

Types of Privileges:

Privilege	Description
SELECT	Read/query data
INSERT	Add new rows
UPDATE	Modify existing data
DELETE	Remove rows
CREATE	Create tables/objects
DROP	Delete tables/objects
ALTER	Modify table structure
EXECUTE	Run stored procedures
ALL	All of the above

DCL vs TCL:

DCL	TCL
Controls user access	Controls transactions
GRANT, REVOKE	COMMIT, ROLLBACK, SAVEPOINT
Who can do what	Whether changes are saved

9. SQL Commands — Complete Reference

Here is a comprehensive overview of all SQL command categories with syntax and examples.
v
visualize show_widget

TCL — Transaction Control Language

TCL manages transactions to maintain database consistency.

sql
```sql
-- COMMIT: permanently save changes
BEGIN;
  UPDATE Account SET balance = balance - 5000 WHERE acc_no = 1001;
  UPDATE Account SET balance = balance + 5000 WHERE acc_no = 1002;
COMMIT;  -- both changes saved permanently
-- ROLLBACK: undo changes since last commit
BEGIN;
  DELETE FROM Student WHERE cgpa < 4.0;
ROLLBACK;  -- deletion undone, data restored
-- SAVEPOINT: set a checkpoint within a transaction
BEGIN;
  INSERT INTO Student VALUES (201, 'Test', '2000-01-01', 'CSE', 7.0, 'test@email.com');
  SAVEPOINT sp1;
  UPDATE Student SET cgpa = 0 WHERE student_id = 201;
  ROLLBACK TO sp1;  -- only rolls back to savepoint, insert is preserved
COMMIT;
```

Joins

sql
```sql
-- INNER JOIN: rows matching in both tables
SELECT s.name, c.course_name
FROM Student s
INNER JOIN Course c ON s.student_id = c.student_id;
-- LEFT JOIN: all rows from left + matching from right
SELECT s.name, e.grade
FROM Student s
LEFT JOIN Enrollment e ON s.student_id = e.student_id;
-- RIGHT JOIN: all rows from right + matching from left
SELECT e.grade, c.course_name
FROM Enrollment e
RIGHT JOIN Course c ON e.course_id = c.course_id;
-- FULL OUTER JOIN: all rows from both tables
SELECT s.name, c.course_name
FROM Student s
FULL OUTER JOIN Course c ON s.student_id = c.student_id;
-- SELF JOIN: join a table with itself
SELECT A.name AS Employee, B.name AS Manager
FROM Employee A, Employee B
WHERE A.manager_id = B.emp_id;
```

Aggregate Functions

sql
```sql
SELECT COUNT(*)        AS total_students    FROM Student;
SELECT SUM(salary)     AS total_salary      FROM Employee;
SELECT AVG(cgpa)       AS average_cgpa      FROM Student;
SELECT MAX(cgpa)       AS highest_cgpa      FROM Student;
SELECT MIN(salary)     AS lowest_salary     FROM Employee;
SELECT branch, COUNT(*) AS count
FROM Student GROUP BY branch;
```

Views

sql
```sql
-- Create a view
CREATE VIEW TopStudents AS
SELECT student_id, name, cgpa
FROM Student
WHERE cgpa >= 8.5;
```

```sql
-- Use the view
SELECT * FROM TopStudents;
-- Drop a view
DROP VIEW TopStudents;
```

Indexes

sql

```sql
-- Create an index (speeds up searches)
CREATE INDEX idx_student_name ON Student(name);
-- Create unique index
CREATE UNIQUE INDEX idx_email ON Student(email);
-- Drop an index
DROP INDEX idx_student_name;
```

Quick Command Summary Table

Category	Command	Purpose
DDL	CREATE	Create table/database
DDL	ALTER	Modify structure
DDL	DROP	Delete table/database
DDL	TRUNCATE	Remove all rows, keep structure
DML	INSERT	Add new data
DML	SELECT	Query/retrieve data
DML	UPDATE	Modify existing data
DML	DELETE	Remove specific rows
DCL	GRANT	Give privileges
DCL	REVOKE	Take back privileges
TCL	COMMIT	Save transaction
TCL	ROLLBACK	Undo transaction
TCL	SAVEPOINT	Set a checkpoint